

PROBLEM DESIGN

Reactive Search System for Dental Supply Inventory Management

1. INTRODUCTION

Background

Fábula Dental is a web-based dental clinic management system that integrates several administrative modules, including patient management, appointments, treatments, payments, and inventory control.

One of the most frequently used modules is the Dental Supplies Inventory Catalog, where the administrator searches among hundreds or thousands of registered supplies, such as anesthetics, dental resins, gloves, disposable materials, and clinical equipment.

As the inventory grows, the user requires an efficient search mechanism capable of displaying matching results instantly, maintaining low response times, and minimizing server resource consumption.

Traditional search implementations generate excessive network requests and introduce synchronization problems that negatively impact both the user experience and server performance.

To address this challenge, this proposal applies the principles of Reactive Programming using RxJS, allowing user input to be processed as an asynchronous stream of events.

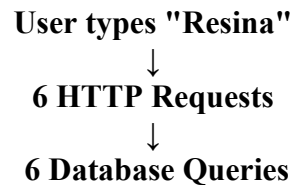
2. PROBLEM STATEMENT

Current Scenario

The Inventory Catalog includes a search bar that allows administrators to locate dental supplies by their names.

Without Reactive Programming, each keyboard event would trigger an immediate filtering process or a request to the backend API. This behavior generates unnecessary processing and network traffic. The proposed reactive implementation prevents these issues by processing the input stream before executing the search.

For example, when searching for "Resina Compuesta", every typed character initiates a new search operation.



Nombre	Cantidad	Costo Unitario	Fecha Pedido	Fecha Caducidad	Estado	Acciones
Resina Compuesta 3M	25	\$12.50	14 ene 2026	14 ene 2028	Vigente	Editar Eliminar

Figure 1. Dental Supplies Inventory Catalog

While this behavior seems acceptable for small datasets, it becomes inefficient as the inventory grows or in a future projection with multiple administrators accessing the system simultaneously.

Identified Problems

- **Excessive Processing:** Every keystroke executes the filtering logic or sends a new request to the backend. This produces unnecessary CPU usage and repeated database operations.
- **Unnecessary API Calls:** If the search is connected directly to the backend, typing a single word may generate multiple HTTP requests.

- **Race Conditions:** Due to network latency, responses may arrive in an unexpected order. If Request A finishes after Request C, outdated results may overwrite the latest search results shown to the administrator.
- **User Experience Issues:** The continuous execution of filtering operations may cause: Interface lag, screen flickering, delayed response, reduced usability.

3. PROPOSED REACTIVE SOLUTION

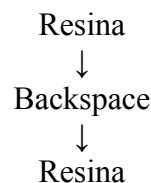
Instead of processing every keyboard event independently, the proposed solution models user input as a stream of asynchronous events using RxJS Observables.

Each keystroke becomes part of an Observable sequence that is processed through reactive operators before interacting with the backend.

This approach processes user input as an Observable stream, applying reactive operators before updating the inventory table. As a result, the application performs searches only after the user pauses typing and ignores duplicate consecutive values, improving responsiveness and reducing unnecessary processing.

Reactive Operators

1. **debounceTime(300):** Waits until the user stops typing for 300 milliseconds before performing the search.
 - a. **Benefits:**
 - Reduces unnecessary processing.
 - Minimizes redundant API requests.
 - Provides a smoother user experience.
 - Improves code maintainability through reactive programming.
2. **distinctUntilChanged():** Avoids executing duplicate searches.



No additional processing occurs because the search term remains unchanged.

3. **map()**: Extracts the current text entered by the administrator from the input field. This value becomes the search term used to filter the dental supplies inventory.
4. **fromEvent()**: Captures every keyboard event generated by the search input and converts it into an Observable stream, allowing the application to react to user input asynchronously.

4.SYSTEM ARCHITECTURE

Components

Frontend

- HTML Inventory View
- Search Input
- Inventory Table
- readController.js
- RxJS (fromEvent, map, debounceTime, distinctUntilChanged)

Backend

- Express.js
- TypeScript
- REST API
- PostgreSQL Database

GET /fabuladental/supplies

Returns:

```
[
  {
    id,
    supplyName,
    quantity,
    unitCost,
    expirationDate,
    status
  }
]
```

Database: Stores all registered dental supplies.

Architecture Diagram

The architecture is divided into two main layers: the Frontend, where RxJS captures and processes user input using reactive operators before executing the search, and the Backend, which exposes the `/fabuladental/supplies` REST endpoint implemented with Express and TypeScript. The API retrieves the inventory data from the PostgreSQL database and returns the matching supplies to update the inventory table.

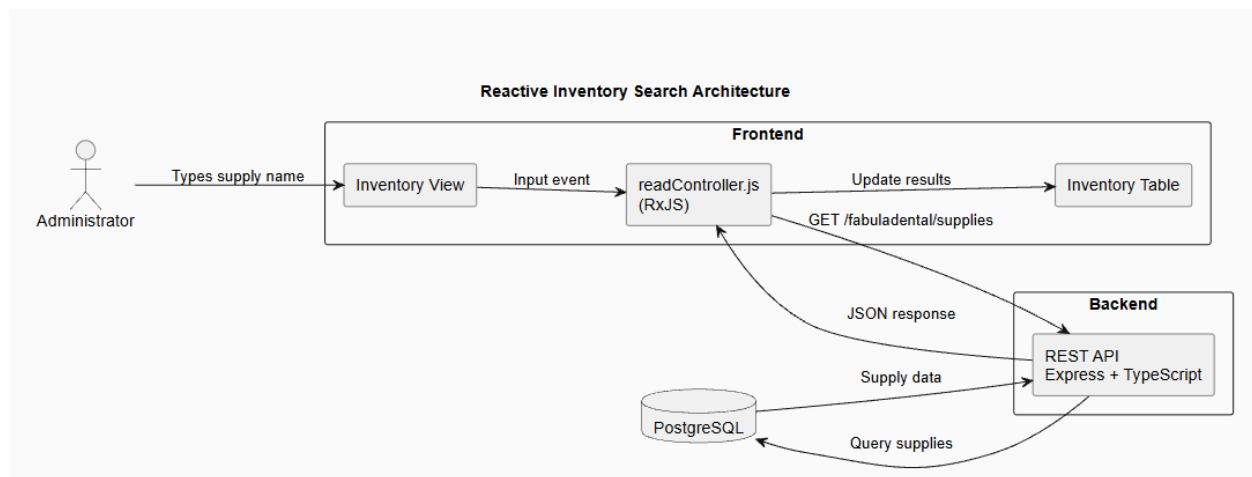


Figure 2. Architecture Diagram for the solution

Use Case Diagram

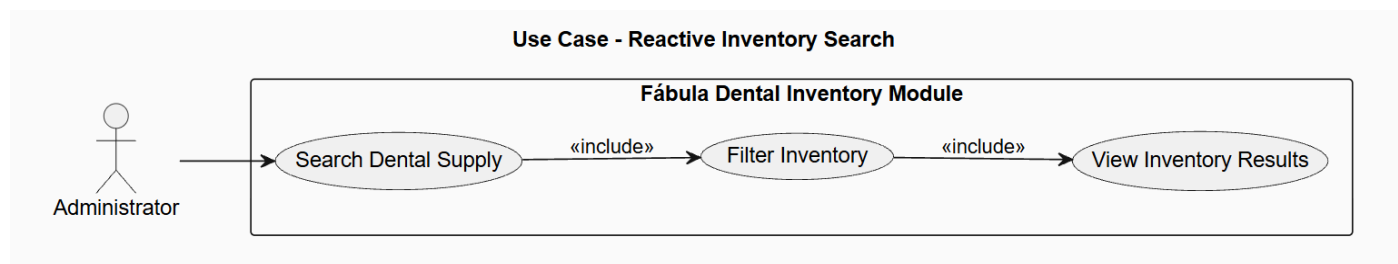


Figure 3. Use Case Diagram for the solution

Activity Diagram

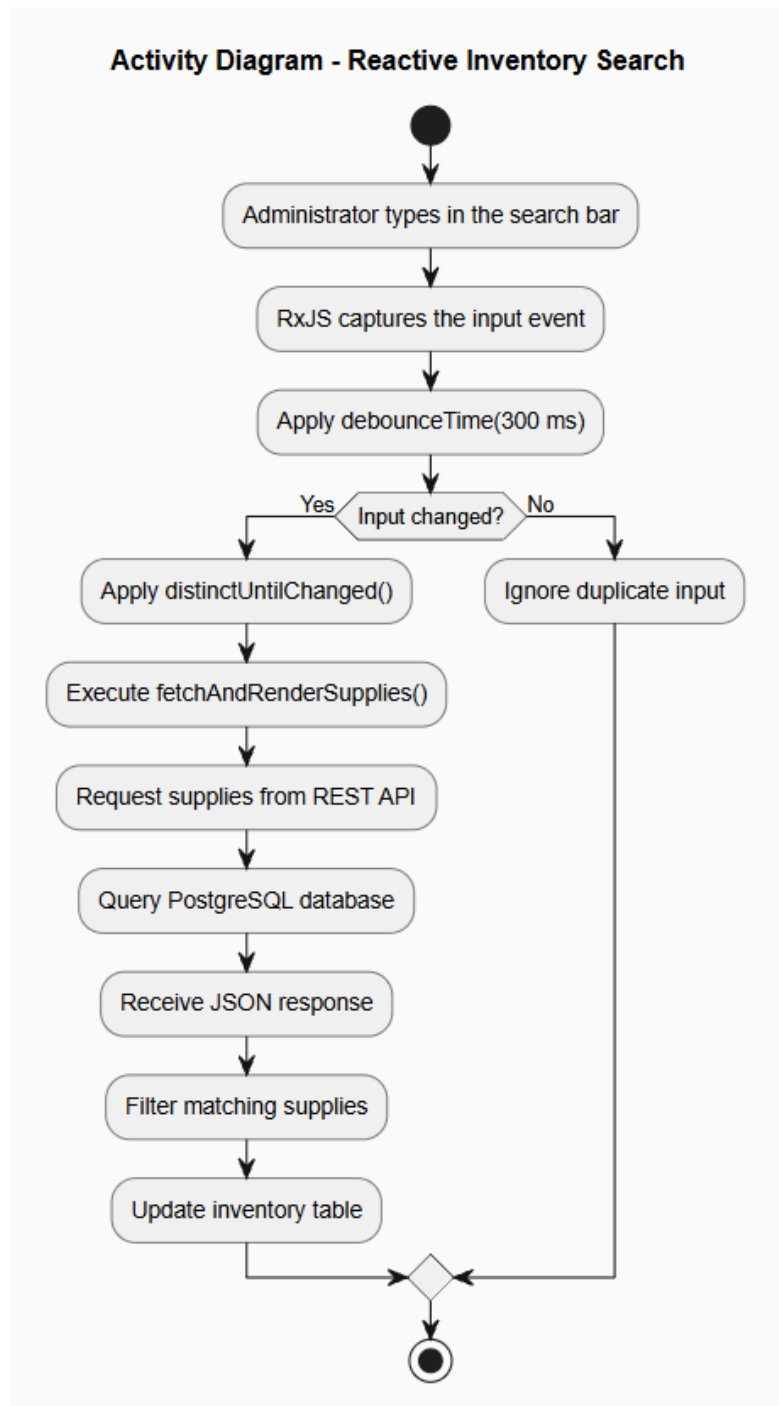


Figure 4. Activity Diagram for the solution

Class Diagram

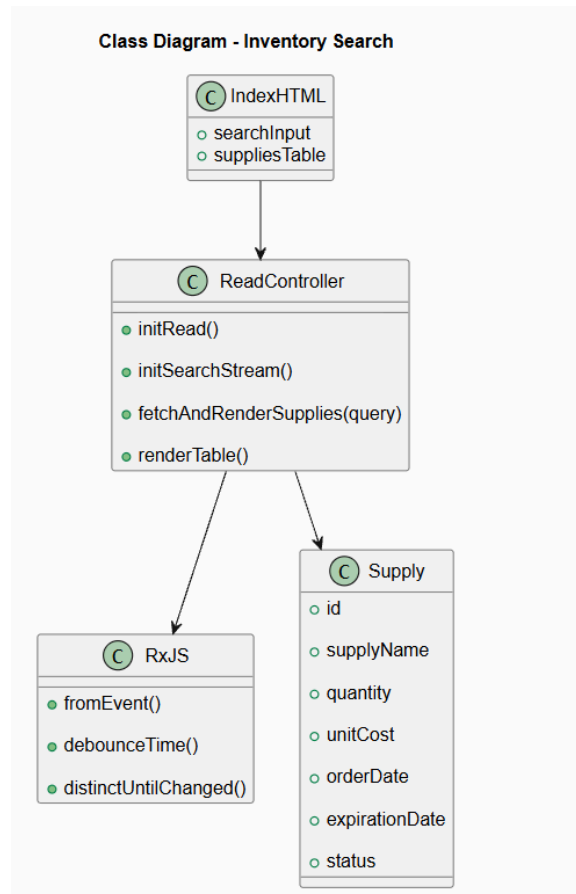


Figure 5. Class Diagram for the solution

Components Diagram

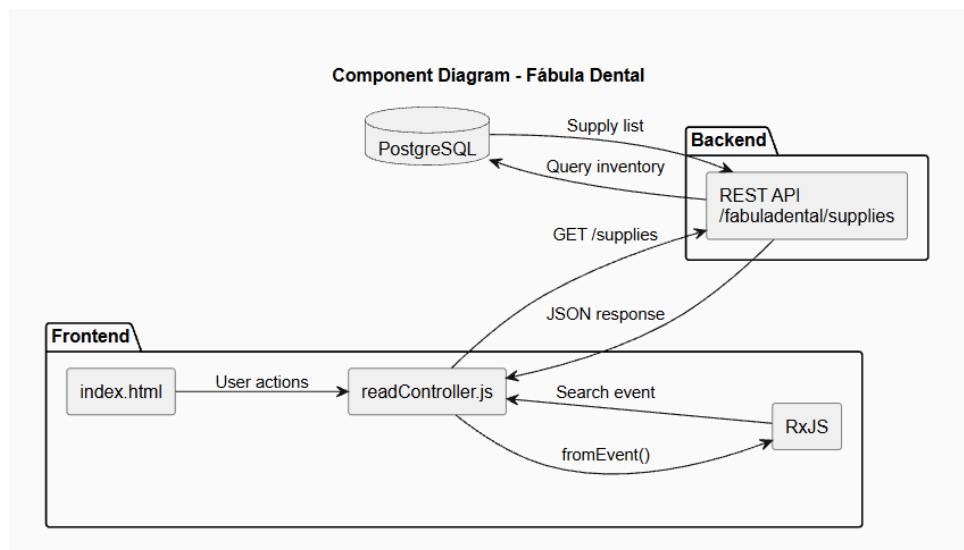


Figure 6. Components Diagram for the solution

9. TECHNOLOGIES USED

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Reactive Library	RxJS
Backend	Node.js + Express
Language	TypeScript
Database	PostgreSQL
API	REST

Table 1. Technology used in the solution

5. CONCLUSION

The proposed solution demonstrates how Reactive Programming can enhance the search functionality of the Fábula Dental Inventory Catalog. By treating user input as an asynchronous stream of events, the system minimizes unnecessary processing, improves application responsiveness, and provides a more reliable and scalable search experience. The use of RxJS operators such as `debounceTime`, `distinctUntilChanged`, and `switchMap` enables an elegant handling of asynchronous events while reducing server workload and preventing inconsistent search results.